

Cryptography with OpenSSL: symmetric encryption and message integrity

Laboratory of the course “Computer system security” (02KRQ)
Politecnico di Torino – AY 2014/15
Prof. Antonio Lioy

prepared by:
Diana Berbecaru (diana.berbecaru@polito.it)

v. 2.35 (17/10/2014)

Purpose of the laboratory

The goal of this laboratory is to experiment with the symmetric cryptography techniques and digest algorithms as well as with the main problems encountered when using these techniques. The laboratory makes use of the OpenSSL tool (<http://www.openssl.org/>), which is a cryptographic suite released as open-source software with BSD license and available for various platforms, including Linux and Windows.

All the exercises proposed use OpenSSL command line programs that provide various cryptographic functions via a specific shell, which you can start with the following command:

```
openssl command [ command_opts ] [ command_args ]
```

Specifically, to run the exercises proposed in this text, you will use the following OpenSSL commands:

```
enc dgst rand speed
```

which will be presented further below. For a complete list of options, as well as for a detailed description of the OpenSSL commands, you can consult the corresponding man pages.

The files and programs needed in some of the exercises proposed in this document are available in the archive at the link: http://security.polito.it/~lioy/02krq/02krq_1415_lab02_support.zip.

OpenSSL enc

The OpenSSL enc command allows to encrypt and decrypt data with several symmetric cipher routines. To view the list of parameters for the command enc, execute the following command:

```
openssl enc -h
```

For a detailed description of the command enc, you can use the command:

```
man enc
```

We provide a short description of the main commands used throughout this laboratory (we remind you that the parameters enclosed by square brackets are optional). The syntax of the command is:

```
openssl enc [-encryption.algorithm] [-e] [-d] [-K key] [-iv vector]  
          [-in input_file] [-out output_file] [-nopad] [-p]
```

where:

- `-encryption_algorithm` is the symmetric encryption algorithm to encrypt and/or decrypt data (e.g. `-aes128`, `-aes-cbc-256`, `-rc4`, the complete list can be found with the command `openssl enc -h`);
- `-e` indicates that the operation to be performed on data is encryption;
- `-d` indicates that the operation to be performed on data is decryption;
- `-K key`, indicates the key to use for the symmetric cryptographic operations (pay attention to use capital letter 'K' not small 'k' for this parameter);
- `-iv vector` indicates the initialization vector to use;
- `-in input_file` indicates the file containing the data to be encrypted or decrypted;
- `-out output_file` indicates the file where to save the result of an encryption or decryption operation;
- `-nopad` indicates that the padding must not be applied (BEWARE: if you use this option with a symmetric block algorithm, the length of the plaintext must be necessarily a multiple of the algorithm block);
- `-p` is used to print on standard output the key and the initialization vector (and also the *salt*, if used).

OpenSSL dgst

The OpenSSL command `dgst` allows to calculate the digest of data using different digest algorithms supported by the OpenSSL library. To view the list of parameters for the command `dgst` execute the command:

```
openssl dgst -h
```

For a detailed description of the `dgst` command, you can use the command:

```
man dgst
```

The syntax of the `dgst` command is:

```
openssl dgst [-digest_algorithm] [-binary] [-hmac key] [-out output_file] [input_file]
```

where:

- `-digest_algorithm` is the digest algorithm to use (e.g. `-md5` to use the MD5 algorithm); the complete list of digest algorithms supported can be found with the command `openssl dgst -h`);
- `-binary` requires output in binary format (by default output is generated in a printable format);
- `-hmac key` indicates to generate as output the HMAC computed with the key `key`;
- `-out output_file` indicates the name of the file where the digest will be saved;
- `input_file` is the file containing the data on which the digest will be calculated (if absent, digest will be calculated on data provided via standard input) – beware that `dgst` uses this form and does not use the option `-in` to specify the file containing the data on which the digest needs to be calculated.

OpenSSL speed

The OpenSSL command `speed` can be used to measure the performance of the various algorithms implemented by OpenSSL. To measure the performance of a specific algorithm, you can use the following command:

```
openssl speed algorithm_name
```

OpenSSL rand

Use the OpenSSL `rand` command to generate `nbyte` pseudo-random data and save them in the file `file_name`:

```
openssl rand -out file_name nbyte
```

Other commands

When executing the exercises, you will have to exchange some data among two computers: for this purpose, you can use the scp utility (acting as client) and the ssh server. Consequently, on one of the two computers you will have to start the ssh server:

```
/etc/init.d/ssh start
```

Subsequently, you can create the kali user on the ssh server (with the command `adduser kali`) and connect remotely with the scp tool to the ssh server.

If you want to connect remotely as root, use the password “toor”.

1 Symmetric cryptography exercises

1.1 Symmetric encryption

In this exercise, you'll use the OpenSSL command `enc` to encrypt a block of data. Create a text file named `ptext` containing the message you want to encrypt, such as “This message is my great secret!”. Note: After creating `ptext`, check that its size is 32 bytes (`ls -l`). Even if it is not absolutely necessary that the length of the plaintext is exactly 32 bytes in this exercise, it will be required in one of the next exercises.

The data contained in the file `ptext` can be encrypted using the AES algorithm with the following command, where the key and the IV will be generated by OpenSSL from a password that you provide and the encrypted text will be saved in the file `ctext`:

```
openssl enc -e -in ptext -out ctext -aes-128-cbc -nosalt -p
```

You should be asked for a password. Pay attention: this password is only used by the OpenSSL command `enc` to generate a key and the initialization vector. You should see something as shown below, even though the key and the IV that you obtain might be different from the ones illustrated below as an example:

```
enter aes-128-cbc encryption password:
Verifying - enter aes-128-cbc encryption password:
key=E358EFA489F58062F10DD7316B65649E
iv =8A222106B38147AB215285A3EEB8990B
```

Find out the corresponding OpenSSL commands used for encrypting the same data with other symmetric algorithms (e.g. DES, 3DES, RC4).

What else do you need to specify when you choose to use DES, 3DES and AES as encryption algorithms? (hint: think whether the size of the data to be encrypted is smaller or bigger than the size of the algorithm block). Write down the possible choices (and commands).

Which is the length of the file `ctext` generated in the various cases (encryption with the various algorithms suggested)? Compare the length of the `ctext` file to the length of the `ptext` file. Do you notice any difference?

In conclusion: what happens when you use the following command to encrypt the message “This message is my great secret!”?

```
openssl enc -e -in ptext -out ctext_aes_128_cbc -K <key> -iv <iv> -aes-128-cbc -nopad -p
```

Try to execute the above command with a different plaintext message. Do you get any error?

What happens when you execute the following command?

```
openssl enc -e -in ptext -out ctext_rc4 -K <key> -rc4 -p -base64
```

Explain the difference among the two commands executed above.

Now find out the commands to decrypt the encrypted message(s) (e.g. files `ctext_rc4` and `ctext_aes128_cbc`) obtained as result of the above commands. Save the decrypted message in a file named `dtext_rc4` and `dtext_aes128_cbc`. Find out the decrypt OpenSSL command(s) and write them down:

Verify whether the decryption has been performed correctly (check out the content of the files `dtext_rc4` and `dtext_aes128_cbc`).

(Optional) Try to generate and use your own key and IV and encrypt the plaintext using the above algorithms.

How does OpenSSL generate the key and the IV in the following command?

```
openssl enc -e -in ptext -out ctext -aes-128-cbc -p
```

1.2 Brute force attack

In this exercise you'll perform a brute force attack against a symmetric cryptographic algorithm!

Form two groups of hosts, named Alessandro and Caterina, and proceeds as follows:

1. Alessandro prepares a new plaintext message and saves it in the file `ptext`. Next Alessandro chooses a key, which is 4 bits long (that is one hexadecimal character) named (i.e. `<short_key>`), and encrypts the message with the following command::

```
openssl enc -e -in ptext -out ctext_ale -K <short_key> -iv 0 -rc4
```

2. Alessandro sends the encrypted message contained in `ctext_ale` to Caterina, by using the following command::

```
scp ctext_ale root@<IPaddress.Caterina>:/root
```

3. Caterina tries to discover the key (with a brute force attack), which was used by Alessandro to encrypt the plaintext message.

Now take the file named `bruteforce_enc`: it was encrypted using the OpenSSL command executed above by Alessandro. Have you encountered (any) difficulties in discovering the correct (encryption) key used? Why?

1.3 Modifying an encrypted message (optional)

Now try to modify an encrypted message. To modify a binary file, use the hexadecimal editor `hexedit`::

```
hexeditor ctext_ale
```

(Press F1 to view the commands for `hexedit`, Ctrl+X to save and exit).

Form three groups of hosts (called Alessandro, Beatrice and Caterina), and proceed in the following manner:

1. Alessandro and Beatrice choose a common (secret) key.
2. Alessandro prepares a new plaintext message containing only decimal digits, and encrypts the plaintext by using the RC4 algorithm and the secret key shared with Beatrice.
3. Alessandro sends then the encrypted message to Caterina. Caterina tries to modify the (encrypted) message before sending it to Beatrice.
4. Beatrice has to find out whether the (encrypted) message received from Caterina was modified or not.

What can do Caterina to modify Alessandro's message, so that the modification goes unnoticed by Beatrice?

What happens when you use AES-128-CBC instead of RC4? (try it).

	100 B	10 kB	1 MB	100 MB
DES-CBC				
DES-EDE3				
RC4				
AES-128-CBC				
AES-256-CBC				
AES-128-IGE				
AES-256-IGE				

Table 1: *Performance of some symmetric encryption algorithms.*

1.4 Operation modes of symmetric block cipher algorithms

In this exercise we'll use the three files `ecb*` available in the archive. The first two files contain two (possible) plaintext messages. The third file contains the ciphertext message obtained by encrypting one of the first two files with the following command:

```
openssl enc -e -in ecb_plain_X -out ecb_enc -K key -iv 0 -des-ecb
```

Can you find out which of the two plaintext messages has been encrypted by comparing the ciphertext message with the two plaintext messages?

1.5 Performance of symmetric encryption algorithms

In this exercise, you will evaluate the time required to perform the cryptographic operations.

Create files of different sizes (e.g. 100 B, 10 kB, 1 MB e 100 MB) using the following command:

```
openssl rand -out r.txt size_in_bytes
```

To evaluate the time required by the encryption operations, you can use the `time` command:

```
time the_openssl_encryption_command
```

Which is the most significant value among the times provided shown by `time`?

Now fill in the Table 1 with the results obtained when encrypting the files with different algorithms and key lengths. Will the decryption time (using the algorithms in the table) be significantly different? (try out).

OpenSSL contains a command used to measure the performance of various cryptographic algorithms. Execute the following command to measure the performance of `aes-128-cbc`:

```
openssl speed aes-128-cbc
```

Which is the difference between using `time` and OpenSSL `speed` command to calculate the performance of an algorithm?

Which one (`time` vs. `speed`) is better to measure the performance of a cryptographic algorithm?

2 Digest algorithms

2.1 Computation and verification of message digests

Create a text message like "This is a trial message to test digest functions!", save it in a file named `msg`, and calculate its digest using MD5, SHA1, SHA-256. Choose the output in binary format and save the results (i.e. the digests) in the files `MD5dgst`, `SHA1dgst` and `SHA256dgst` respectively. Find out the correct OpenSSL commands to calculate the digests and write them down:

Now try to modify the message (e.g. delete the “!” at the end of the message) and recalculate the digest. What do you note when you compare the (above) two digests (you can use a hexeditor for this purpose)?

Suppose now that you want to calculate the hash of `msg` by using the the AES (symmetric) algorithm: how can you do this operation (think at the AES mode to use, the key, the IV if necessary)?

Compare the above approach with the one performed by the utility `crypt` (for linux platforms):

```
man crypt
```

2.2 Collisions

In this exercise, you’ll evaluate the robustness of a digest algorithm based on digest length generated by the digest algorithm. For this purpose, we will present an example digest algorithm that we implemented on purpose for this exercise - called Insecure Hash Algorithm (IHA1). Our insecure digest algorithm (whose implementation in C is shortly shown further below) makes a XOR bit by bit (one byte at a time) of the data whose digest need to be calculated and returns as output a hash value on 4 bits:

```
while ((xc=fgetc(file_in)) != EOF)
    hash ^= xc;
printf ("iha1(%s) = %X\n", argv[1], hash % 16);
```

The file `iha1.c` is included in the archive containing the support material for this laboratory. You can compile the program with the following command:

```
gcc iha1.c -o iha1.exe
```

The file `iha1.exe` takes as the first parameter from the command line the name of the file containing the data on which the “insecure hash” will be calculated:

```
./iha1.exe input_file
```

Create a file containing the following message M: “This is a test message” and calculate its digest with IHA1.

Now you can try to find a collision: can you create another message M’ which gives the same digest (as the one calculated above) considering that the IHA1 algorithm is used to calculate also the digests on the message M’?

How many IHA1 operations are required (on average) to find the collision, that is to perform the above operation (remember that the digest is 4-bits long)?

Suppose you want to find out an arbitrary collision, that is find any two messages whose digest are the same (in this case, the digest value to be matched is not fixed as above). How long should be the digest so that the time required to find the arbitrary collision is equal to the time required to find the collision above in which the digest value was fixed.

2.3 Application of digest algorithm: keyed digest

Suppose Alessandro wants to generate a keyed-digest of a message that he wants to send to Beatrice. Alessandro creates a text message named `msg` such as:

```
This is a test message used to create a keyed digest!
```

and creates a file `key` in which he stores the key he will use to calculate the keyed-digest.

How can Alessandro use the OpenSSL `dgst` command to calculate a keyed-digest on `msg`? What Beatrice needs to know in order to be able to verify the keyed digest? Which are the steps Beatrice needs to perform to verify the keyed digest received from Alessandro?

	100 B	10 kB	1 MB	100 MB
MD5				
SHA1				
RMD160				
SHA256				
SHA512				

Table 2: *Costs associated with some digest algorithms.*

Describe two possible applications of the keyed digest created above.

Find out the OpenSSL command that can be used to calculate an HMAC keyed-digest.

How can we verify the HMAC keyed-digest calculated with the previous OpenSSL command?

Suppose you want to calculate a keyed-digest on `msg` by using the AES algorithm used in the previous exercises: what technique can you use?

2.4 Application of digest algorithm: file integrity (optional)

The tools `sha1sum` and `sha1deep` allow to compute easily the hash of one or more files (the second one processes recursively the files contained in a directory).

Create your own directory named `tree`, together with a subtree of directories and files; for simplicity you could use the test script named `gen_tree.sh`.

Write down the command used to calculate the digest of all files contained in the directory `tree`.

Form two groups of hosts (called Alessandro and Beatrice) and proceed as follows:

1. Alessandro connects to the PC of Beatrice via SSH and modifies some files in the directory `tree`.
2. Beatrice has to discover which files have been modified by Alessandro.

What can do Alessandro to modify (some) files so that Beatrice does not discover the modification?

What can do Beatrice to protect herself from the above attack? Enumerate at least two methods:

2.5 Performance

Evaluate the cost associated to the digest algorithms implemented by OpenSSL, by using the method explained/used in Exercise 1.5. Fill in the results in the Table 2.

Compare the results obtained in this exercise with the ones obtained for the symmetric encryption algorithms.

3 Putting all together

How can you use the OpenSSL commands described in this text to send a confidential document to a peer? How can you use the OpenSSL commands to send a document to a peer so that she can verify its authenticity and integrity?